

API Documentation

Base URL: <http://127.0.0.1:8000>

Authentication

All protected endpoints require a JWT Bearer token:

```
Authorization: Bearer <access_token>
```

1. Authentication APIs

POST /api/auth/register/

Request Body

```
{
  "username": "Kushal",
  "email": "kushal@gmail.com",
  "password": "Password123"
}
```

Response — 201 Created

```
{
  "id": 2,
  "username": "Kushal",
  "email": "kushal@gmail.com"
}
```

POST /api/auth/login/

Request Body

```
{
  "username": "Kushal",
  "password": "Password123"
}
```

Response — 200 OK

```
{
  "refresh": "eyJhbGciOiJIUzI1NiIs... ",
  "access": "eyJhbGciOiJIUzI1NiIs..."
}
```

POST /api/auth/token/refresh/

Request Body

```
{
  "refresh": "eyJhbGciOiJIUzI1NiIs..."
}
```

2. Product APIs

GET /api/products/

Public endpoint. Optional query: ?category=1 (includes child categories via DFS).
Response: 200 OK — array of products.

GET /api/products/1/

Public endpoint — returns a single product's details.

POST /api/products/

Admin only. Requires Authorization: Bearer <admin_token>.

Request Body

```
{
  "name": "Laptop",
  "sku": "LAP-001",
  "description": "High performance laptop",
  "price": "50000.00",
  "stock": 20,
  "status": true,
  "category": 1
}
```

GET /api/products/categories/1/tree/

Returns the category tree via depth-first search.

Response

```
{
  "category_id": 1,
  "children": [
    "Laptops",
    "Phones",
    "Gaming Laptops"
  ]
}
```

GET /api/products/recommendations/1/

Returns related products from the same category tree (DFS).

3. Order APIs (JWT Required)

GET /api/orders/

Lists the authenticated user's orders.

POST /api/orders/

Request Body

```
{
  "items": [
    {
      "product": 1,
      "quantity": 2
    },
    {
```

```
        "product": 2,
        "quantity": 1
      }
    ]
  }
```

Note: Do not send "user" in the body — it is auto-assigned from the JWT.

Response — 201 Created

```
{
  "id": 1,
  "user": 1,
  "total_amount": "100800.00",
  "status": "pending",
  "items": [
    {
      "id": 1,
      "product": 1,
      "quantity": 2,
      "price": "50000.00",
      "subtotal": "100000.00"
    },
    {
      "id": 2,
      "product": 2,
      "quantity": 1,
      "price": "800.00",
      "subtotal": "800.00"
    }
  ]
}
```

GET /api/orders/1/

Returns details for a single order belonging to the authenticated user.

4. Payment APIs (JWT Required)

GET /api/payments/

Lists the authenticated user's payments.

POST /api/payments/create/

Request Body — Stripe

```
{
  "order": 1,
  "provider": "stripe"
}
```

Request Body — bKash (mock)

```
{
  "order": 1,
  "provider": "bkash"
}
```

```
}
```

Response — 201 Created

```
{
  "id": 2,
  "order": 1,
  "provider": "stripe",
  "transaction_id": "pi_3Tt0C9HCohIAyvBv1IaJLHey",
  "status": "pending",
  "raw_response": { ... }
}
```

GET /api/payments/2/

Returns details for a single payment.

5. Webhook APIs (No JWT)

POST /api/payments/stripe/webhook/

Called by Stripe when a payment succeeds or fails. Test locally with the Stripe CLI.

POST /api/payments/bkash/webhook/

Request Body

```
{
  "transaction_id": "BKASH-XXXXXXXXXXXX",
  "status": "success"
}
```

Common Error Responses

Code	Meaning
401	Missing or expired JWT token — login again
403	Not authorized (e.g., non-admin creating product)
404	Resource not found
400	Validation error (missing fields, insufficient stock)

Postman Tips

1. **Wrong URL format:** use GET `http://127.0.0.1:8000/api/payments/` — do not type GET inside the URL bar.
2. **Expired token:** if you see "Token is expired", login again and update the Bearer token.
3. **Orders need items:** send at least one product in the items array when creating orders.
4. **Payments endpoint:** use POST `/api/payments/create/` to start a payment, not POST `/api/payments/`.

Import the Postman collection from [docs/postman_collection.json](#).